

架构辩论赛记录

本实验严格按照要求，由4名成员分别采用不同 Prompt 策略向 AI 发起架构咨询，完整记录 Prompt 与 AI 回答，并开展 30 分钟团队讨论。

成员 A：直接提问（无约束、无背景）

Prompt

推荐什么架构

AI 回答

对于一般的信息管理系统，我推荐使用**微服务架构**。微服务架构具备良好的扩展性、独立部署能力、高可用性，适合业务持续迭代，服务之间低耦合，便于团队分工开发和维护。

成员 B：提供详细项目约束后提问

Prompt

我们是 4 人学生团队，要在 10 周内完成裁判文书论证标注系统，前端用 Vue，后端用 Spring Boot，请问推荐什么架构？

AI 回答

结合你们**团队规模小、工期短、技术栈固定、课程项目**的特点，我推荐 **MVC 架构** 或 **模块化垂直架构**。这两种架构上手快、结构简单、学习成本低，不需要复杂部署与治理，非常适合在短时间内完成可用版本。

成员 C：要求 AI 对比多种方案的优劣

Prompt

请对比**分层架构、模块化垂直架构、微服务架构**这三种方案，在学生课程项目、4 人团队、10 周工期的场景下的优劣。

AI 回答

- **微服务架构**：复杂度极高，需要服务拆分、网关、注册中心、配置等，10 周完全不可能完成，**不推荐**。
 - **分层架构**：结构清晰，但对设计要求高，层间容易耦合，学生团队易混乱，**中等推荐**。
 - **模块化垂直架构**：按业务功能拆分，每人负责一块，前后端一起写，耦合最低、开发最快、最易验收，**最推荐**。
-

成员 D：让 AI 扮演架构师角色进行分析

Prompt

你是一名有 10 年经验的资深架构师，请为裁判文书论证标注系统设计架构。已知条件：4 人学生团队、10 周交付、课程项目、技术栈 Vue+Spring Boot、需求明确、无需高并发。

AI 回答

作为架构师，我给出的最佳方案是：

模块化垂直架构 + 前后端分离

理由如下：

1. 按业务功能划分为独立模块，每人负责一个完整功能，分工明确不冲突。
 2. 前后端分离适配 Vue+Spring Boot，接口统一，并行开发效率高。
 3. 结构简单、调试容易、文档易写，非常适合课程验收。
 4. 无需分布式、无需中间件，10 周内可稳定完成全部核心功能。
-

团队辩论会记录

会议基本信息

- 会议主题：架构方案辩论与最终决策
- 参与人员：A、B、C、D（4 人团队全体）
- 会议时长：30 分钟

- 会议形式：线上讨论
 - 讨论目标：基于 4 种不同 Prompt 得到的 AI 建议，对比方案优缺点，确定最终架构
-

一、成员依次展示与发言

成员 A 展示

- 展示内容：直接提问 "推荐什么架构"，AI 推荐**微服务架构**。
- 个人分析：该回答完全脱离项目实际，只强调扩展性、高可用、独立部署等技术优点，完全没有考虑我们是 4 人学生团队、只有 10 周工期。微服务涉及服务拆分、网关、注册中心、配置、链路追踪等大量内容，学习成本极高，我们根本不可能在短时间内完成开发与调试。该建议不具备可行性，只能作为技术了解。

成员 B 展示（带项目约束提问）

- 展示内容：提供团队规模、工期、技术栈（Vue + Spring Boot）后，AI 推荐 **MVC / 模块化架构**。
- 个人分析：加入约束后，AI 明显变得更务实，给出的方案更贴近课程项目。AI 明确指出了 "开发速度快、学习成本低、适合小团队" 等关键点，和我们的需求匹配度较高。但回答缺少具体的模块划分思路，只给出架构名称，指导性有限。

成员 C 展示（多方案对比）

- 展示内容：让 AI 对比分层、模块化、微服务三种架构。
- 个人分析：AI 给出了非常清晰的优劣判断：微服务直接排除；分层架构中等；模块化最优。这个回答逻辑性强、维度全面，明确点出 "学生团队、工期短、复杂度可控" 是核心约束，让我们快速排除了不合理方案。但内容偏向理论，缺少具体落地方式。

成员 D 展示（扮演资深架构师）

- 展示内容：AI 以架构师身份给出**模块化垂直架构 + 前后端分离**，并给出详细理由：分工清晰、并行开发、接口统一、快速交付。
 - 个人分析：该回答最完整、最落地、最贴合工程实际。不仅给出架构，还解释了 "为什么适合我们"，包括开发模式、分工方式、接口规范、工期优势。这是最能直接用于课程项目的方案，可执行性最强。
-

二、团队自由讨论

讨论要点 1: 不同 Prompt 对 AI 输出质量的影响

- 成员一致认为:

1. 直接提问 = 低质量、泛化、不可用

AI 只提供通用理论, 不结合场景, 给出的建议脱离现实。

2. 加入约束 = 回答明显变精准

告诉 AI 人数、工期、技术栈后, 答案立刻务实。

3. 要求对比 = 帮助快速排除错误方案

让 AI 做对比能降低我们的决策成本。

4. 角色设定 + 详细背景 = 输出质量最高

架构师角色让 AI 思考更全面、更工程化, 给出可落地的完整方案。

讨论要点 2: 分层架构 vs 模块化垂直架构

- 分层架构优点: 结构标准、层次清晰。
- 分层架构缺点: 层间耦合难控制, 学生团队容易写出 "混乱分层", 前期设计时间长, 10 周内风险较高。
- 模块化垂直架构优点: 按业务功能拆分, 每人负责一个模块的前后端, 分工明确、不打架、开发快、便于验收。
- 结论: **模块化垂直架构更适合我们的团队与工期。**

讨论要点 3: 是否考虑 MVC?

- 讨论结论:

MVC 可以用, 但更适合页面为主的简单系统。

我们的标注系统有复杂交互、三栏布局、论证图展示, **模块化垂直架构更利于功能解耦**。因此最终不单独使用 MVC, 而是采用**模块化垂直 + 前后端分离**。

三、讨论总结

1. 直接提问的 AI 回答几乎无实际价值，太空泛、过于理想化，不考虑工程约束。
 2. Prompt 越详细、背景越完整、角色越清晰，AI 给出的架构建议越可靠。
 3. 模块化垂直架构在开发效率、分工清晰性、可行性、学习成本上全面占优。
 4. 前后端分离是现代 Web 项目标准模式，配合 Vue + Spring Boot 非常合适。
 5. 微服务、复杂分层架构均不适合本项目的规模与工期。
-

四、团队投票结果

投票主题：是否确定采用 模块化垂直架构 + 前后端分离 作为最终架构？

- 同意：A、B、C、D（4 票）
- 反对：0 票
- 弃权：0 票

最终决议：全票通过。

五、最终架构确认说明

最终选定架构

模块化垂直架构 + 前后端分离

选择理由

1. 按业务功能拆分模块，分工清晰，互不干扰，便于 4 人并行开发。
2. 学习成本低，开发速度快，能确保 10 周内高质量交付。
3. 模块独立，便于调试、测试、撰写文档与课程验收。
4. 接口统一，前后端分离清晰，适配 Vue3 + Spring Boot 技术栈。
5. 风险低，结构简单，符合学生团队能力范围。

不选择其他方案的原因

- **不选分层架构：**设计要求高、边界难维护、前期耗时长，10 周内风险高。

- **不选微服务架构：**复杂度极高、学习成本大、部署困难，完全超出课程项目范围与能力。
- **不选单体 MVC：**对复杂标注业务的扩展性、分工友好度不如模块化垂直架构。